

UNIT 4

Ensemble Techniques And Unsupervised Learning: Combining multiple learners: Model combination schemes, Voting, Ensemble Learning - bagging, boosting, stacking, Unsupervised learning: Introduction to Clustering, K-means clustering, Dimensionality reduction-Principal Component Analysis

Bias and Variance in Machine Learning

- **Bias and Variance** are two major sources of error that affect a machine learning model's performance
- Understanding them helps in designing models that **generalize well to unseen data**.
- Balance bias and variance to achieve **optimal model performance**.

Bias

- Bias refers to errors due to overly simplistic assumptions in the learning algorithm
- Models with high bias tend to underfit the data, meaning they fail to capture the underlying patterns in the dataset.
- Bias measures **how far the model's average prediction is from the true value.**

Example

- Suppose the true relationship is: $y=x^2$
- But the model you use is **linear regression**: $y=ax+b$
- A straight line **cannot represent a quadratic curve**, so the model will consistently make errors.

Characteristics of High Bias

- Model is **too simple**
- **Underfitting**
- Training error is **high**
- Validation/test error is **also high**

Variance

- Variance measures the model's sensitivity to fluctuations in the training data.
- Variance measures **how much the model's predictions change if the training data changes slightly**.
- If a model learns **noise in the data instead of the real pattern**, it has **high variance**.

Characteristics of High Variance

- Model is **too complex**
- **Overfitting**
- Training error **very low**
- Test error **high**

Ensemble Learning

- **Ensemble technology** in machine learning refers to techniques that **combine multiple models to produce better performance than any single model alone.**
- The main idea is that a group of "weak learners" (models that perform slightly better than random guessing) can be combined to form a "strong learner."
- Ensembles offer more accuracy than individual or base classifier.
- Ensemble methods can **decrease variance using bagging approach, bias using a boosting approach, or improve predictions using stacking approach.**

Real life examples

- Let's take a real example to build the intuition.
- Suppose, you want to invest in a company XYZ. You are not sure about its performance though.
- So, you look for advice on whether the stock price will increase by more than 6% per annum or not?
- You decide to approach various experts having diverse domain experience

The survey prediction

- Employee of Company XYZ:
 - In the past, he has been right 70% times.
- Financial Advisor of Company XYZ:
 - In the past, he has been right 75% times.
- Stock Market Trader:
 - In the past, he has been right 70% times.
- Employee of a competitor:
 - In the past, he has been right 60% times.
- Market Research team in the same segment:
 - In the past, he has been right 75% times.
- Social Media Expert:
 - In the past, he has been right 65% times.

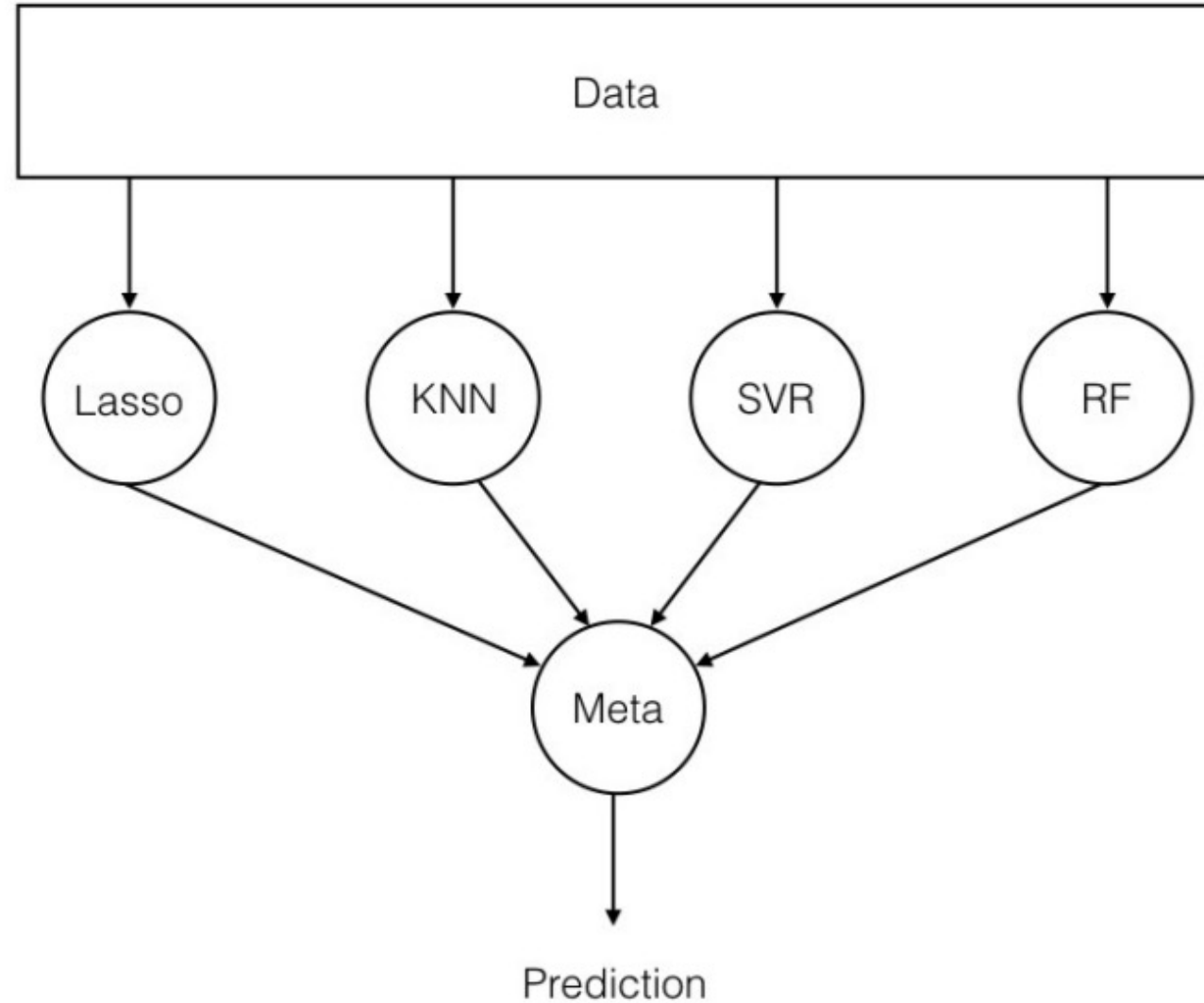
Inference

- Given the broad spectrum of access you have, you can probably combine all the information and make an informed decision.
- In a scenario when all the 6 experts/teams verify that it's a good decision (assuming all the predictions are independent of each other), you will get a combined accuracy rate of $1 - (30\% \cdot 25\% \cdot 30\% \cdot 40\% \cdot 25\% \cdot 35\%) = 1 - 0.07875 = 99.92125\%$
- The assumption used here that all the predictions are completely independent is slightly extreme as they are expected to be correlated. However, you can see how we can be so sure by combining various forecasts together.
- Well, Ensemble learning is no different.

Ensembling

- An ensemble is the art of combining a diverse set of learners (individual models) together to improvise on the stability and predictive power of the model.
- In our example, the way we combine all the predictions collectively will be termed as Ensemble learning.
- Moreover, Ensemble-based models can be incorporated in both of the two scenarios, i.e., when data is of large volume and when data is too little.

Basic Ensemble Structure



Summary

- Use multiple learning algorithms (classifiers)
- Combine the decisions
- Can be more accurate than the individual classifiers
- Generate a group of base-learners
- Different learners use different
 - Algorithms
 - Hyperparameters
 - Representations (Modalities)
 - Training sets

Why ensembles ?

- There are two main reasons to use an ensemble over a single model, and they are related; they are:
 - **Performance**: An ensemble can make better predictions and achieve better performance than any single contributing model.
 - **Robustness**: An ensemble reduces the spread or dispersion of the predictions and model performance.

Ensemble Learning Methods

- Bagging
- Boosting
- Stacking
- Voting

Bootstrapping

Bootstrapping is a technique where we **create many new datasets from the original dataset by sampling randomly with replacement.**

- Each bootstrap sample is likely to have **some repeated and some missing records.**
 - These samples are then used to train different models.
-
- It helps us understand the variability in our model and improves stability and accuracy when used in ensemble learning,

Bootstrapping

Original Dataset: [1, 2, 3, 4, 5]

You create bootstrap samples like

- Sample 1: [1, 3, 4, 4, 5]
- Sample 2: [2, 2, 3, 5, 1]
- Sample 3: [5, 3, 1, 3, 4]

Bagging or Bootstrap Aggregation

- Bagging, an abbreviation for Bootstrap Aggregating, is a machine learning ensemble strategy for enhancing the reliability and precision of predictive models.
- Bagging involves training multiple base learners on different subsets of the training data, typically created through random sampling with replacement.
- It can be used for both classification and regression

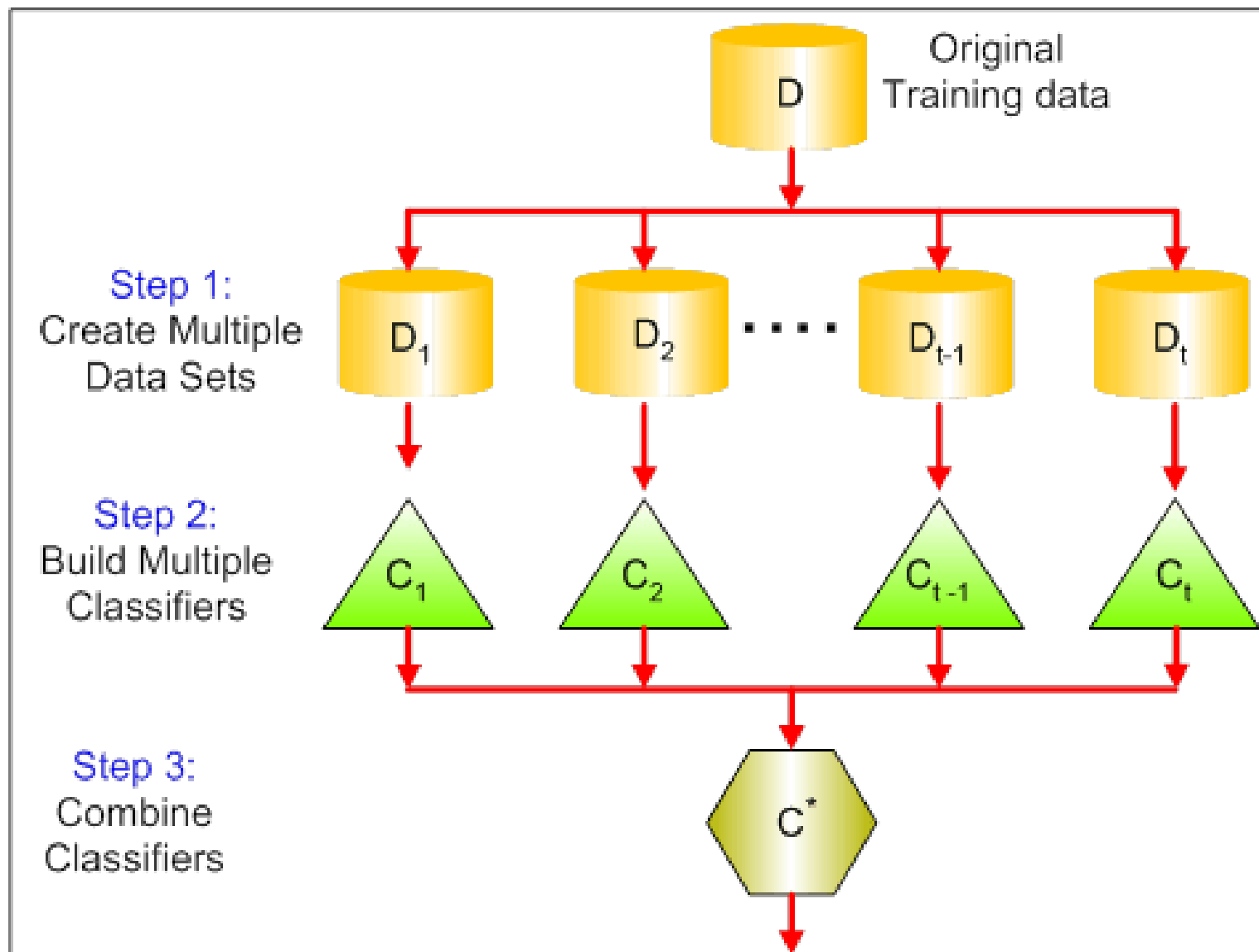
Bagging (Bootstrap Aggregating):

- **Goal:** Reduce **variance** and avoid **overfitting**.
- **How it works:**
 - Take **many random samples** (with replacement) from the original dataset.
 - Train a separate model on each sample.
 - **Average** (for regression) or **vote** (for classification) the predictions.
- **Popular example: Random Forest** (many decision trees).

Variance

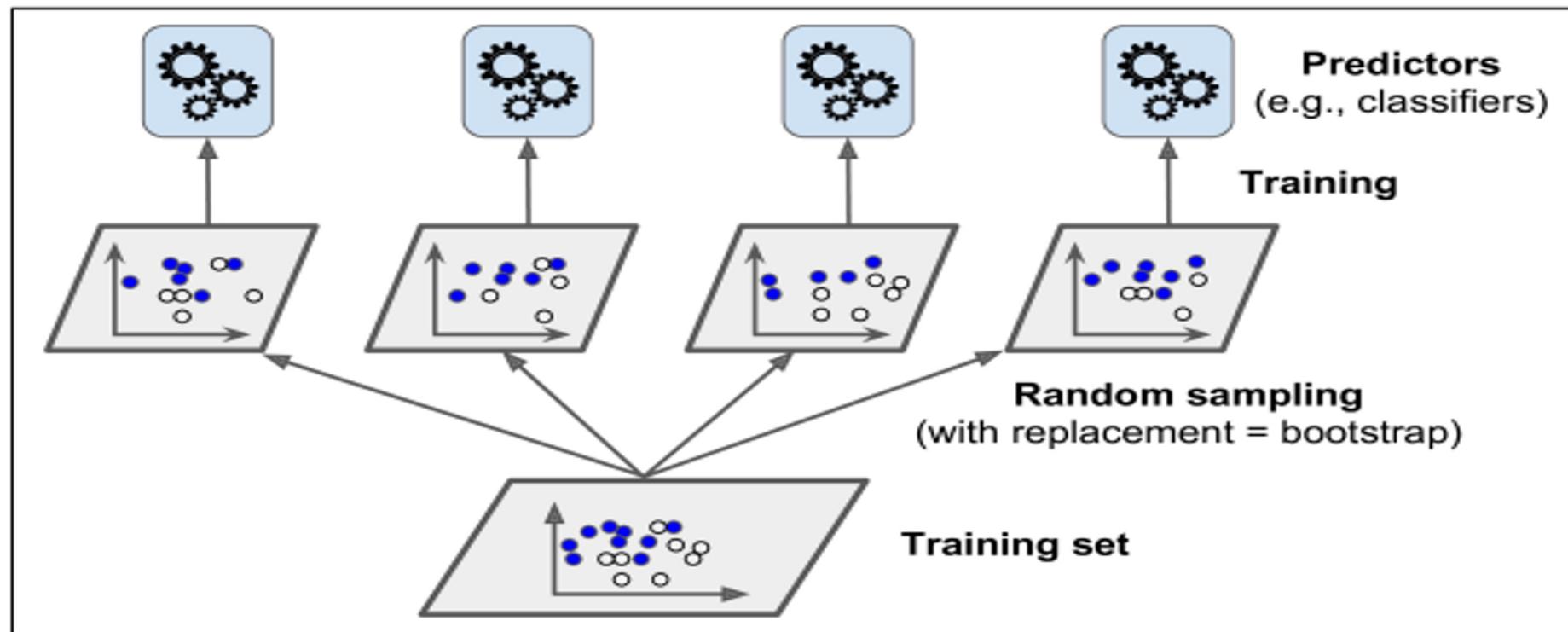
Variance means **how much your model's predictions change** if you use a **different training dataset**.

- **High variance** = Model is **too sensitive** to the training data.
 - It **learns noise** or **unnecessary details** (overfitting).
 - Performs **well on training data** but **poorly on new/unseen data**.
- **Low variance** = Model is **more stable**.
 - Even if the training data changes a little, predictions stay **similar**.



- Multiple base models are trained independently and in parallel on different subsets of the training data.
- Each subset is generated using bootstrap sampling, in which data points are picked at random with replacement.
- In the case of the bagging classifier, the final prediction is made by aggregating the predictions of the all-base model using majority voting.
- In the models of regression, the final prediction is made by averaging the predictions of the all-base model, and that is known as bagging regression.
- Bagging helps improve accuracy and reduce overfitting, especially in models that have high variance.

- Use the same training algorithm for every predictor, but to train them on different random subsets of the training set.
- When sampling is performed with replacement, this method is called bagging (short for bootstrap aggregating).
- When sampling is performed without replacement, it is called pasting.



Random Forest

- Random forest is a type of supervised machine learning algorithm based on **ensemble learning**.
- **Random forest** combines **multiple decision trees** to make more accurate and stable predictions.
- The random forest algorithm combines multiple algorithm of the same type i.e. multiple decision trees, resulting in a forest of trees, hence the name "Random Forest".
- The random forest algorithm can be used for both regression and classification tasks.

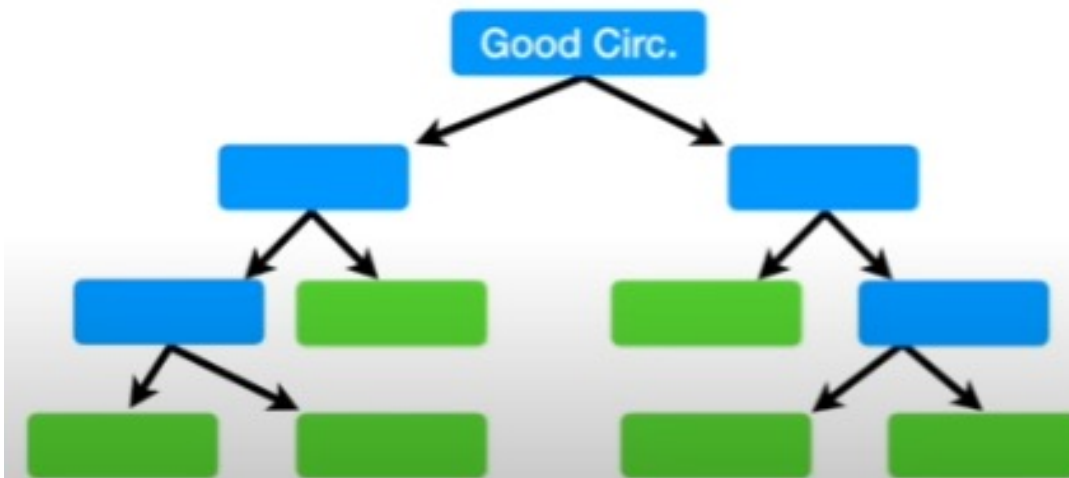
Random Forest

- **Bootstrapping** – Each tree is trained on a different random sample of the training data (with replacement).
- **Feature Randomness** – At each split in a tree, a random subset of features is considered.
- **Ensemble of Trees** – All trees vote, and the majority wins.

Random Forest

We built a tree...

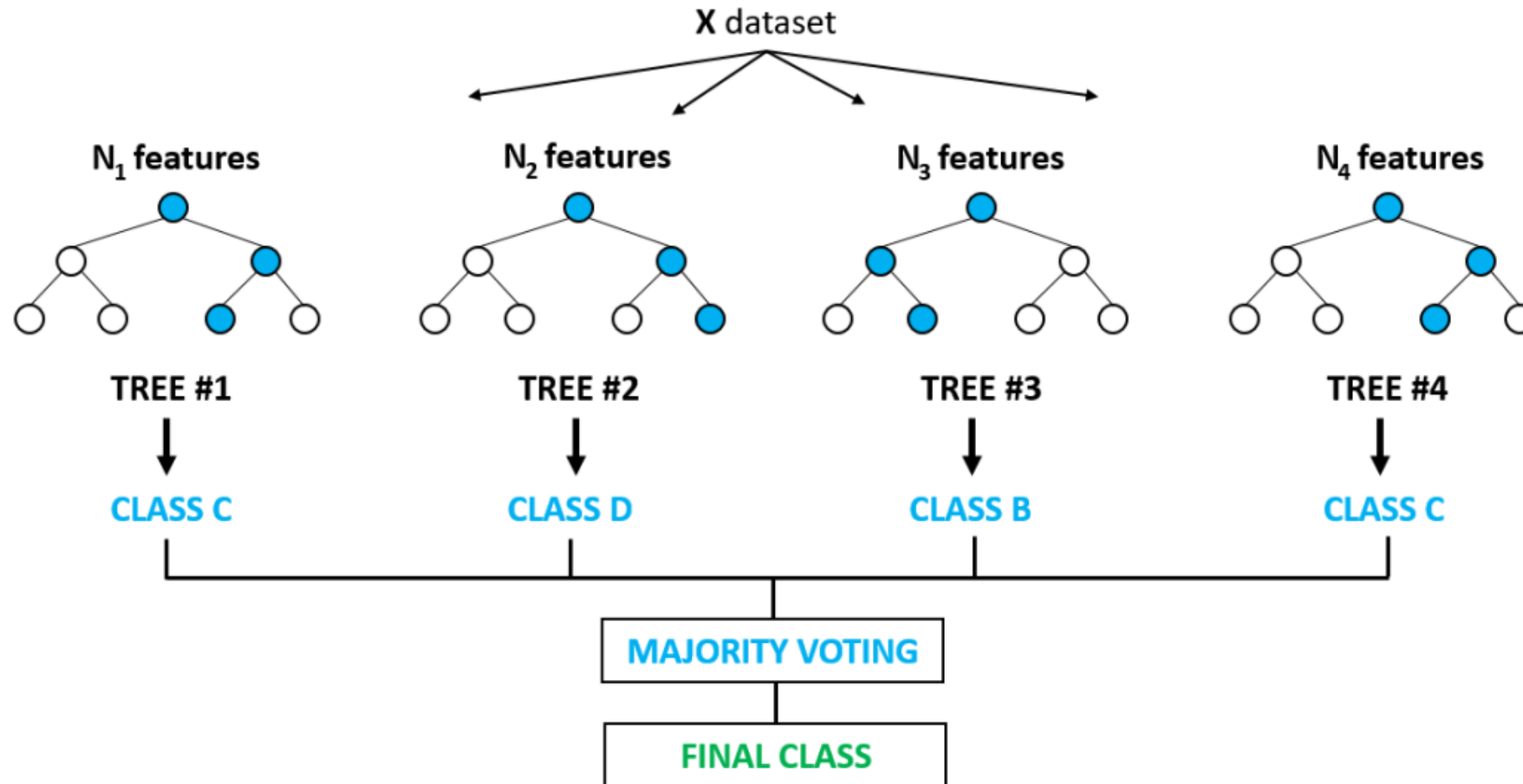
- 1) Using a bootstrapped dataset
- 2) Only considering a random a subset of variables at each step.



Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

Majority voting



Random Forest

- Pick N random records from the dataset.
- Build a decision tree based on these N records.
- Choose the number of trees you want in your algorithm and repeat steps 1 and 2.
- In case of a regression problem, for a new record, each tree in the forest predicts a value for Y (output). The final value can be calculated by taking the average of all the values predicted by all the trees in forest. Or, in case of a classification problem, each tree in the forest predicts the category to which the new record belongs. Finally, the new record is assigned to the category that wins the majority vote.

Boosting

- Boosting improves performance by training models sequentially.
- Each new model tries to correct errors made by the previous one.
- This is done by building a model from the training data, then creating a second model that attempts to correct the errors from the first model. Models are added until the training set is predicted perfectly or a maximum number of models are added.
- Highly accurate classifier(or strong classifier) offer error rate close to 0.
- Boosting algorithms are less affected by the overfitting problem.

Boosting

- **Goal:** Reduce **bias** (and a little bit of variance).
- **How it works:**
 - Train models **sequentially**.
 - Each model **focuses more** on the mistakes made by the previous model.
 - Combine all models into one strong model (weighted sum or voting).
- **Popular examples: AdaBoost, Gradient Boosting, XGBoost.**

Boosting Models

Models that are typically used in Boosting technique are:

- XGBoost (Extreme Gradient Boosting)
- GBM (Gradient Boosting Machine)
- ADABOOST (Adaptive Boosting)

AdaBoost

- **AdaBoost** is a **boosting ensemble machine learning algorithm** that combines multiple **weak learners** (usually decision stumps) to create a **strong classifier**.
- The idea is
 - Instead of training all models equally, **AdaBoost focuses more on the samples that previous models classified incorrectly**.
 - So, each new model **learns from the mistakes of the previous model**.

Weak Learner

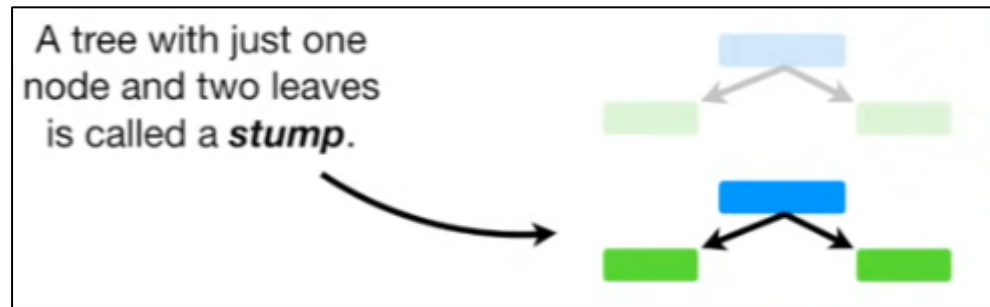
- A model slightly better than random guessing.
- Usually a **decision stump** (tree with depth = 1).

Sample Weights

- Each training sample has a **weight**.
- Initially all weights are **equal**.
- Misclassified samples get **higher weight** in the next iteration

Model Weight (α)

- Each weak learner is assigned a weight based on its accuracy

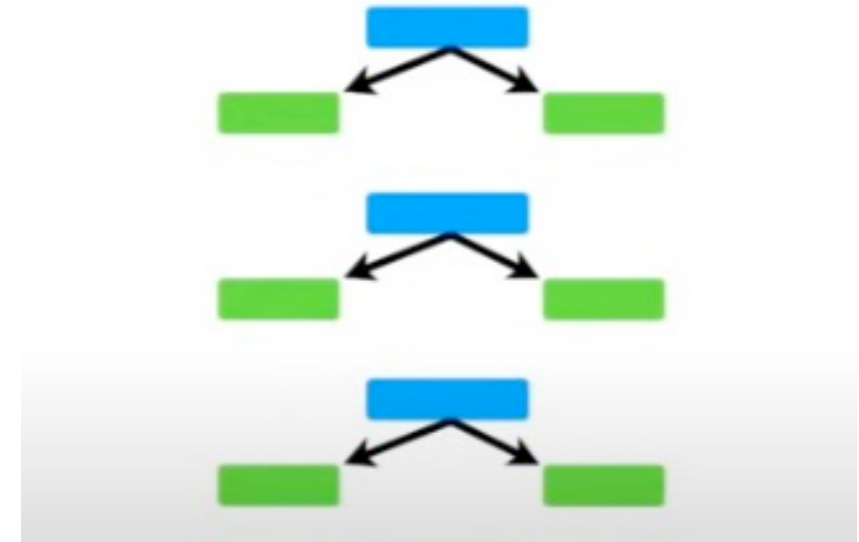


Adaboost

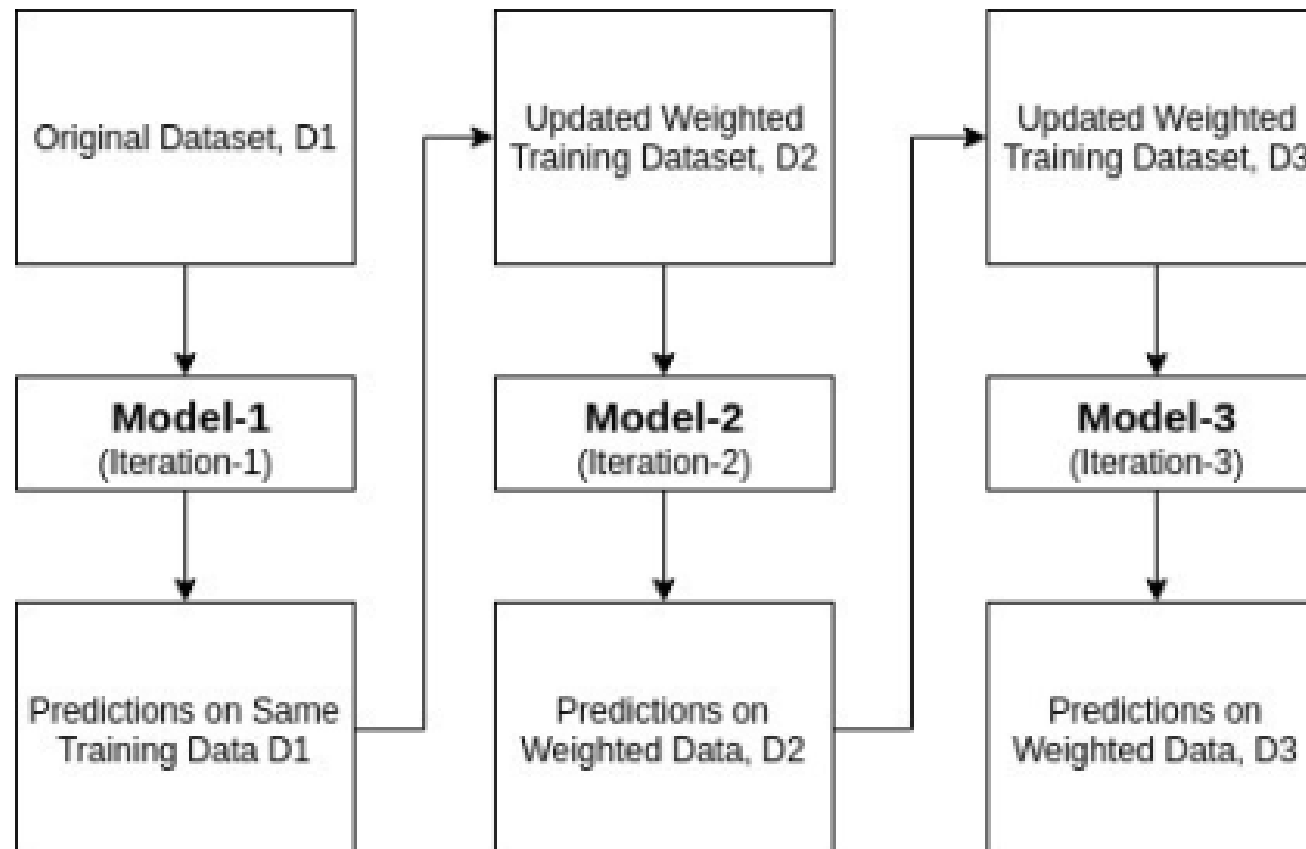
AdaBoost works in **iterations**

- Start by giving **equal weight** to all training samples.
- Train a **weak learner** (e.g., decision stump).
- Check which samples were **misclassified**.
- **Increase weights** of misclassified samples.
- Train the next learner using the updated weights.
- Combine all learners into a **weighted majority vote**.

In contrast, in a **Forest of Trees** made with **AdaBoost**, the trees are usually just a **node** and two **leaves**.



Working Principle of Adaboost



Training Dataset

Sample	x	y	Number of samples
1	1	+1	$N = 5$
2	2	+1	Initial weights
3	3	-1	$w_i = \frac{1}{5} = 0.2$
4	4	-1	
5	5	+1	

Sample	x	y	Weight
1	1	+1	0.2
2	2	+1	0.2
3	3	-1	0.2
4	4	-1	0.2
5	5	+1	0.2

ITERATION 1

Step 1: Train Decision Stump

Consider stump

$x < 4.5 \rightarrow -1$

$x \geq 4.5 \rightarrow +1$

Predictions

Sample	x	True y	Prediction
1	1	+1	-1
2	2	+1	-1
3	3	-1	-1
4	4	-1	-1
5	5	+1	+1

Step 2: Weighted Error

Misclassified samples

- Sample 1
- Sample 2

$$error = 0.2 + 0.2$$

$$error = 0.4$$

Step 3: Classifier Weight

$$\begin{aligned}\alpha_1 &= \frac{1}{2} \ln \left(\frac{1 - error}{error} \right) \\ &= \frac{1}{2} \ln \left(\frac{0.6}{0.4} \right) \\ &= \frac{1}{2} \ln(1.5) \\ \alpha_1 &= 0.203\end{aligned}$$

Update Weights

Formula

$$w_i^{new} = w_i e^{-\alpha y_i h_i(x_i)}$$

Compute $y_i h_i(x_i)$

Sample	y	h(x)	(y h(x))
1	+1	-1	-1
2	+1	-1	-1
3	-1	-1	+1
4	-1	-1	+1
5	+1	+1	+1

Misclassified samples (1,2)

$$\begin{aligned} w &= 0.2 e^{0.203} \\ &= 0.2(1.225) = 0.245 \end{aligned}$$

Correct samples (3,4,5)

$$\begin{aligned} w &= 0.2 e^{-0.203} \\ &= 0.2(0.816) = 0.163 \end{aligned}$$

Normalized weights

Sample	Weight
1	0.245
2	0.245
3	0.163
4	0.163
5	0.163

So samples 1 and 2 now have highest importance.

Now we train using **updated weights**.

Consider stump

$$x < 2.5 \rightarrow +1$$

$$x \geq 2.5 \rightarrow -1$$

Ultimately, the patient is classified
as **Has Heart Disease** because
this is the larger sum.

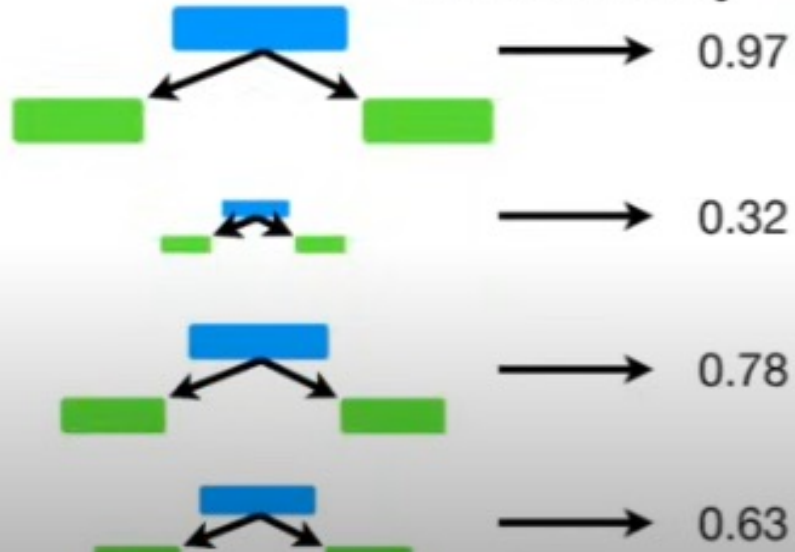
Has Heart Disease

Total = 2.7

Total = 1.23

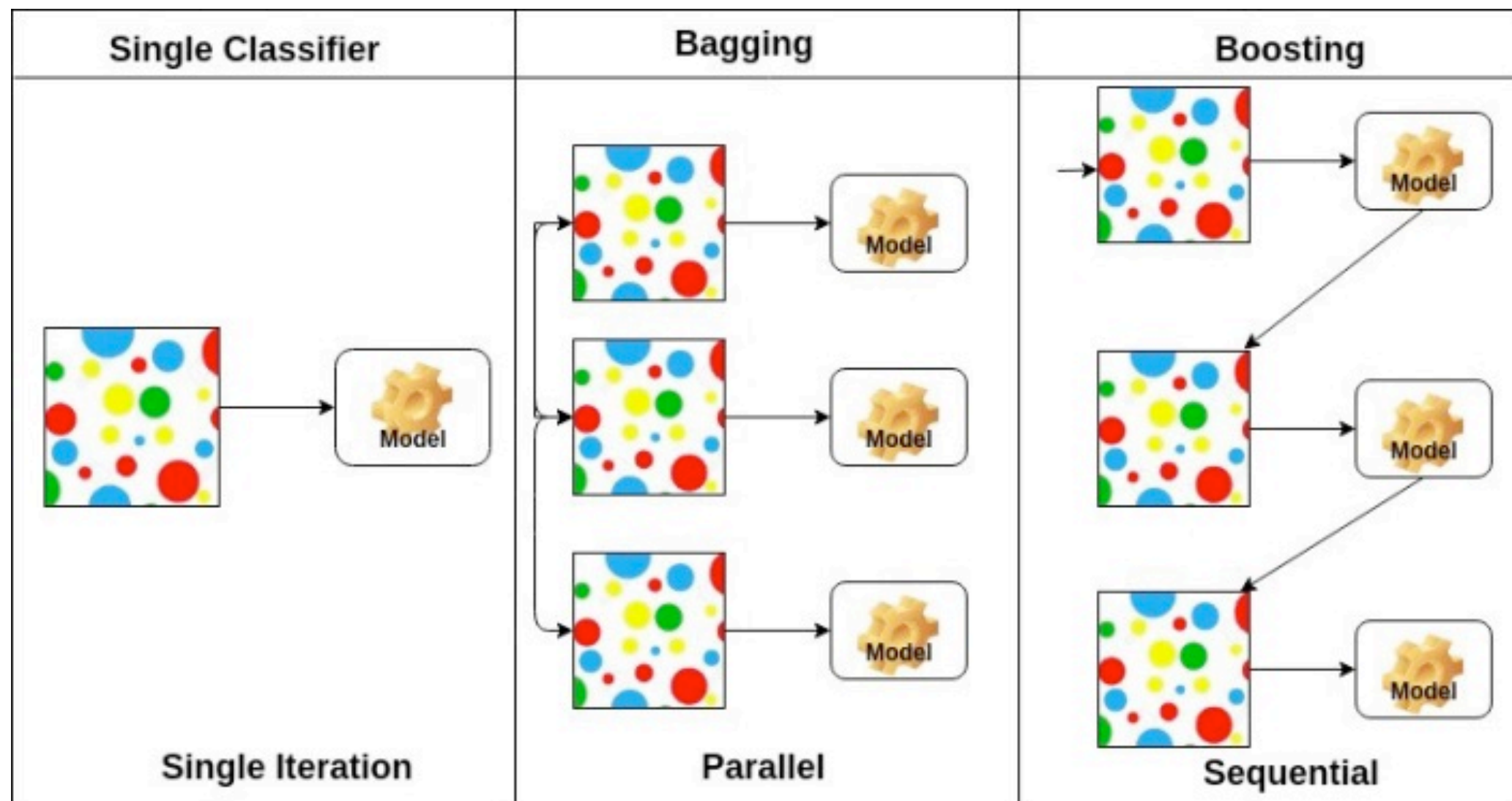
Does Not Have
Heart Disease

Amount of Say



Amount of Say

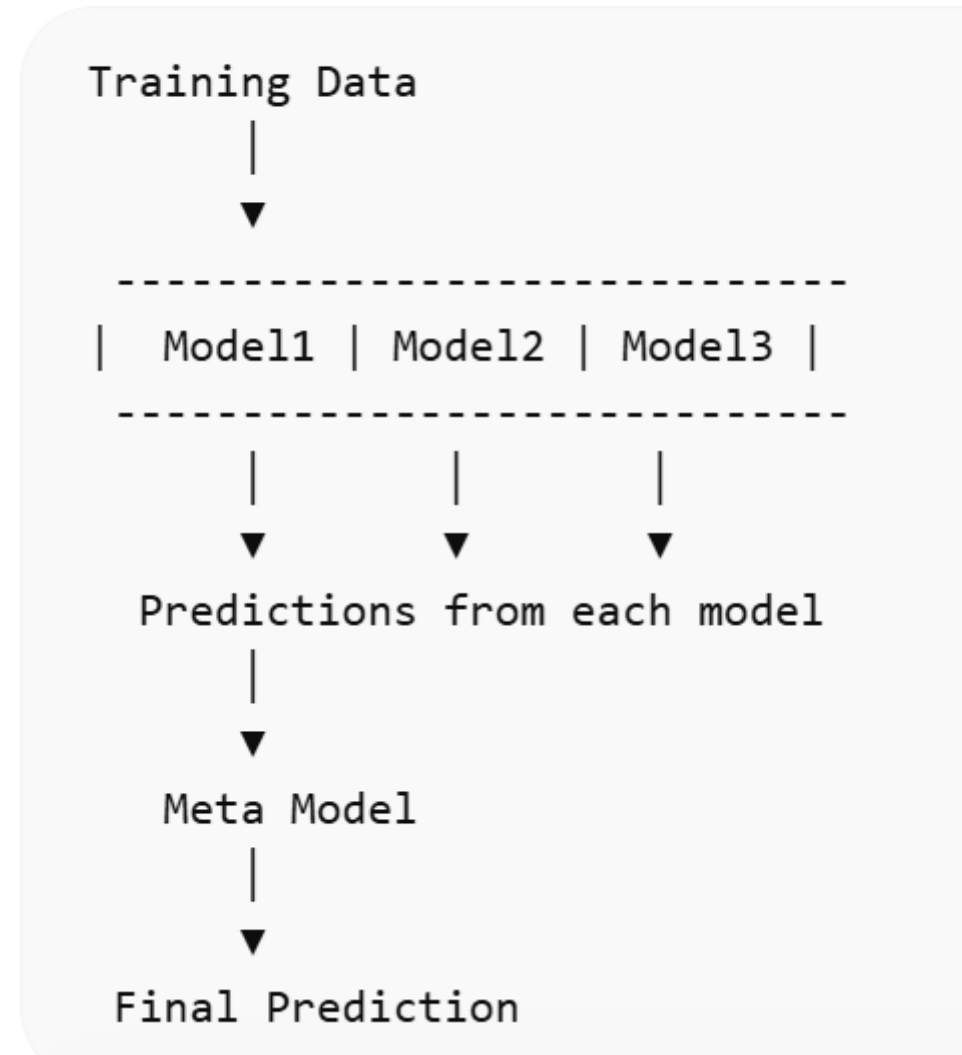




Stacking

- **Stacking** is an **ensemble learning technique** where multiple different models are trained, and their predictions are used as inputs to another model (called a **meta-model**) that learns how to combine them.
- So instead of simple voting, stacking **learns the best way to combine models**.

Conceptual Architecture



Suppose we want to predict **whether a student will pass an exam.**

Student	Study Hours	Attendance	Pass (y)
1	2	60	0
2	3	65	0
3	5	70	1
4	6	80	1
5	7	85	1

Step 1 — Train Base Models

We train three models:

- **Decision Tree**
- **SVM**
- **Logistic Regression**

Step 2 — Base Model Predictions

After training, each model predicts outcomes.

Student	Tree	SVM	Logistic	True y
1	0	0	0	0
2	0	1	0	0
3	1	1	1	1
4	1	1	1	1
5	1	1	1	1

- The above predictions become Meta Dataset for Meta Model
- Train Meta Model on this new dataset.

Train Meta Model

Suppose the meta model is **logistic regression**.

The model tries to learn

$$y = f(\text{Tree}, \text{SVM}, \text{Logistic})$$

For logistic regression:

$$z = w_1(\text{Tree}) + w_2(\text{SVM}) + w_3(\text{Logistic}) + b$$

Then

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

During training the algorithm learns **weights** w_1, w_2, w_3 .

Final Prediction

New data point predictions:

Tree	SVM	Logistic
1	0	1

Meta model computes

$$z = 0.7(1) + 0.2(0) + 0.6(1)$$

$$z = 1.3$$

Prediction

$$P(y = 1) = 0.79$$

Final output = **1**

Voting

- Voting combines multiple models' outputs by majority (classification) or average (regression).
- It can be hard voting (final decision) or soft voting (based on predicted probabilities).
- In classification, a hard voting ensemble involves summing the votes for crisp class labels from other models and predicting the class with the most votes.
- A soft voting ensemble involves summing the predicted probabilities for class labels and predicting the class label with the largest sum probability.

Voting

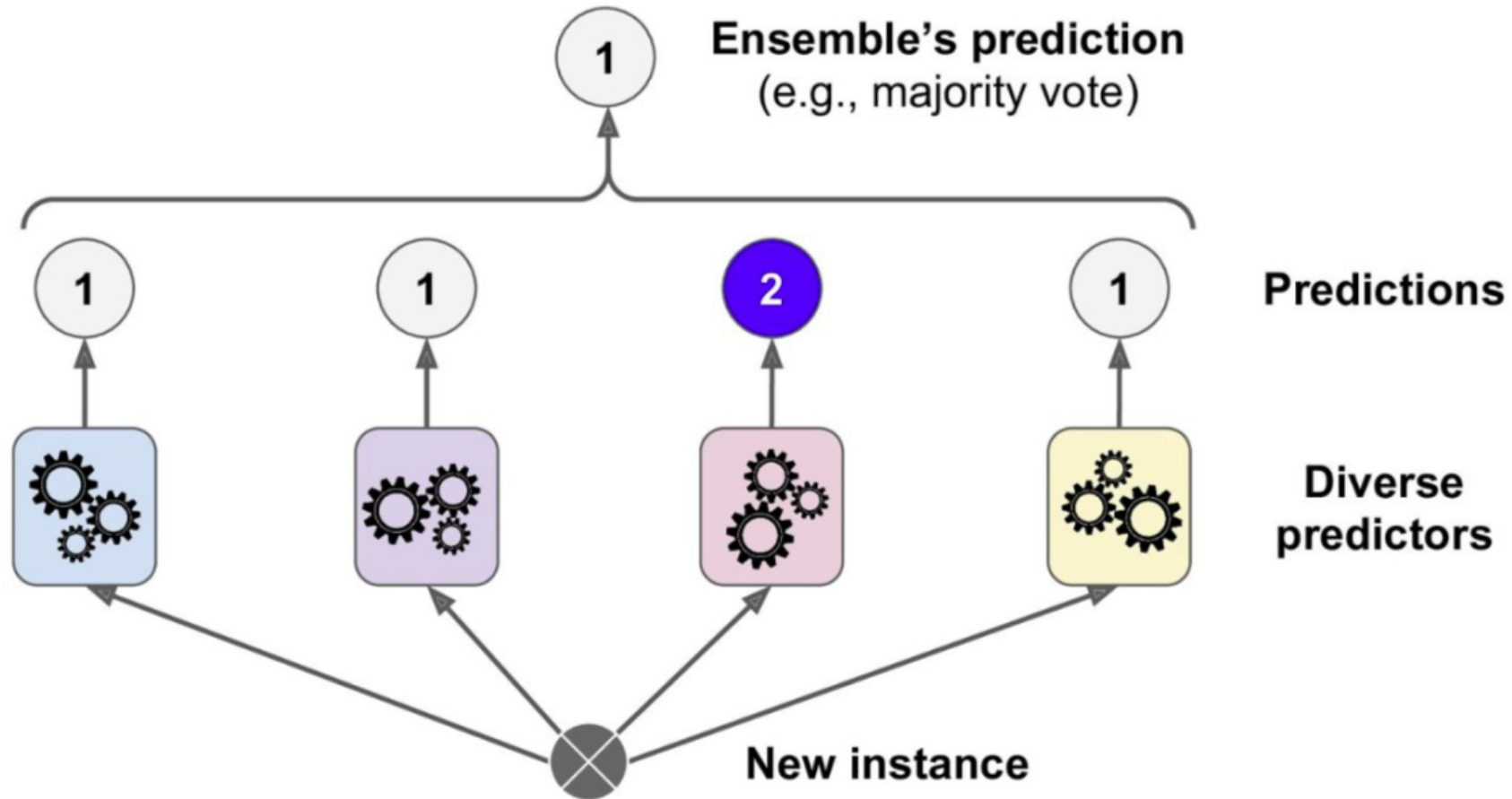


Figure 7-2. Hard voting classifier predictions

Soft voting in Classification

In **Soft Voting**, multiple classifiers predict **probabilities** for each class. The **average (or weighted average)** of these probabilities is used to decide the final class.

Let's say we have 3 classifiers predicting whether a patient has a disease (**Yes = 1, No = 0**). Here's how they assign **probabilities**:

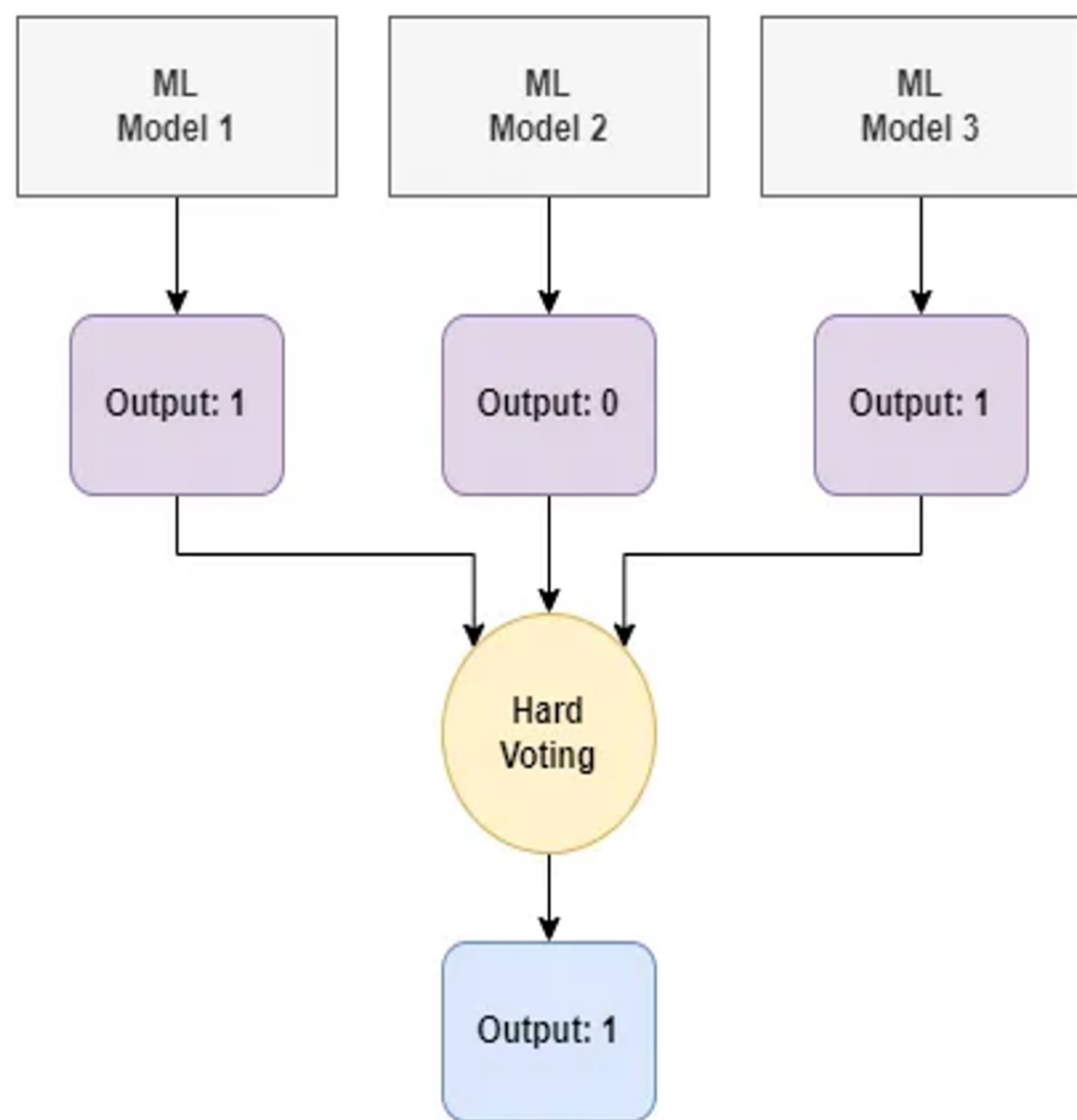
Classifier	Prob(Yes)	Prob(No)
Model A	0.90	0.10
Model B	0.70	0.30
Model C	0.40	0.60

- Average Prob(Yes) = $(0.90 + 0.70 + 0.40) / 3 = \mathbf{0.67}$

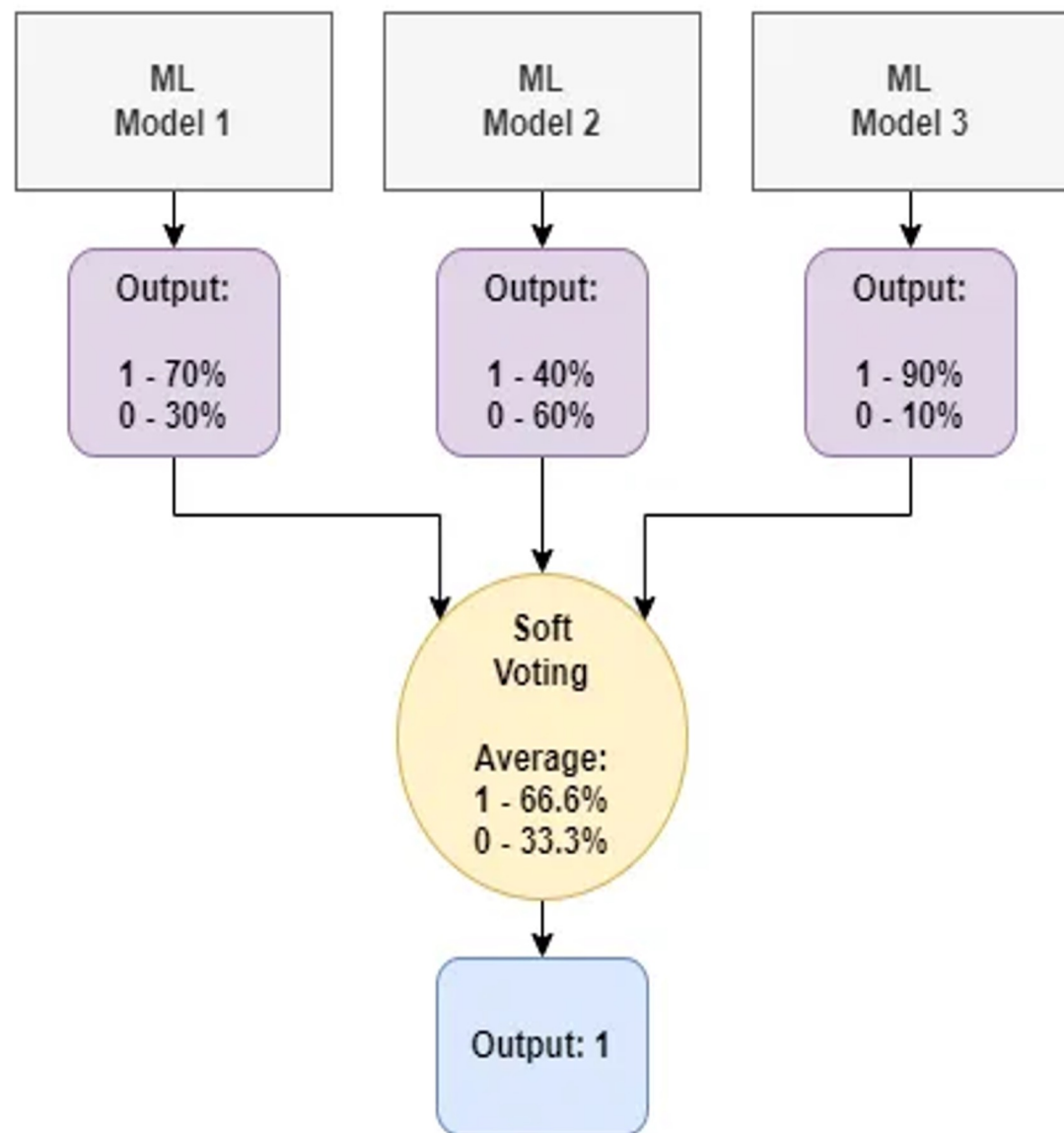
- Average Prob(No) = $(0.10 + 0.30 + 0.60) / 3 = \mathbf{0.33}$



Final prediction: **Yes (1)** because $0.67 > 0.33$.



Depiction of Hard Voting in Ensemble Machine Learning



Depiction of Soft Voting in Ensemble Machine Learning

Unsupervised learning

- **Unsupervised learning** is a type of machine learning where the model learns patterns from **unlabeled data** — meaning, the data doesn't have predefined categories or outputs.
- Instead of being told what the "right answer" is (like in supervised learning), the model tries to:
 - **Discover hidden structures** (e.g., clusters, groups)
 - **Find patterns, similarities, or differences**
 - **Reduce the dimensionality** of the data to make it easier to understand

Example

If you give an unsupervised model a lot of pictures of animals without labels, it might figure out by itself that some images are "cats" and others are "dogs" — purely based on patterns like size, shape, fur texture, etc

Real-World Problem

Organizing a photo album

Customer segmentation in business

Detecting unusual credit card usage

Genetics research

Unsupervised Learning Application

Clustering similar faces/images together

Grouping customers based on buying behavior

Anomaly detection (flagging strange transactions)

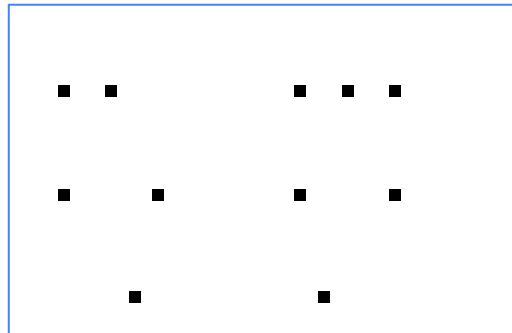
Finding hidden structures in DNA sequences

Common techniques in unsupervised learning:

- **Clustering** (e.g., K-Means, DBSCAN) — grouping similar data points
- **Dimensionality reduction** (e.g., PCA, t-SNE) — simplifying data without losing important information
- **Anomaly detection** — identifying unusual data points
- **Association rules** (e.g., Market Basket Analysis) — finding relationships between variables

Introduction to Clustering

- **Clustering** is a type of **unsupervised learning** where the goal is to **group similar data points together** based on their features — **without** having any predefined labels.



A clustering algorithm would group the left side dots into one cluster and the right-side dots into another cluster — without you telling it anything!

How clustering works (basic idea)

- You give the model a set of points.
- The model measures **how close** or **similar** the points are.
- It **groups** points that are **more alike** into clusters.

Common Clustering Algorithms

Algorithm

What it does

K-Means

Divides data into 'k' groups, tries to minimize the distance inside each group

Hierarchical Clustering

Builds a tree of clusters based on distance

DBSCAN

Groups closely packed points and marks others as noise

Mean Shift

Finds clusters by shifting points toward the highest data density

Applications

- **Document Classification:** Using k-means clustering, we can divide documents into various clusters based on their content, topics, and tags.
- **Customer segmentation:** Supermarkets and e-commerce websites divide their customers into various clusters based on their transaction data and demography. This helps the business to target appropriate customers with relevant products to increase sales.
- **Cyber profiling:** In cyber profiling, we collect data from individuals as well as groups to identify their relationships. With k-means clustering, we can easily make clusters of people based on their connection to each other to identify any available patterns.
- **Image segmentation:** We can use k-means clustering to perform image segmentation by grouping similar pixels into clusters.
- **Fraud detection in banking and insurance:** By using historical data on frauds, banks and insurance agencies can predict potential frauds by the application of k-means clustering.

K-MEANS CLUSTERING

- K-means clustering is an unsupervised machine learning algorithm used to group data points into clusters based on their similarity
- The algorithm works by iteratively assigning data points to the nearest cluster centroid (The mean of the data points within each cluster, serving as the center of the cluster) , and then recalculating the centroid positions until the clusters stabilize
- The goal is to minimize the distance between data points and their assigned cluster centroids.

K-MEANS CLUSTERING

1. Initialization

- The algorithm starts by randomly selecting k data points as initial cluster centroids.

2. Assignment:

- Each data point is assigned to the closest centroid, creating k clusters.

3. Update

- The centroids are recalculated as the mean of all data points assigned to each cluster.

4. Iteration:

Steps 2 and 3 are repeated until the cluster assignments no longer change significantly.

K-means Clustering Numerical Example

Point	Coordinates
A1	(2,10)
A2	(2,6)
A3	(11,11)
A4	(6,9)
A5	(6,4)
A6	(1,2)
A7	(5,10)
A8	(4,9)
A9	(10,12)
A10	(7,5)
A11	(9,11)
A12	(4,6)
A13	(3,10)
A14	(3,8)
A15	(6,11)

We are also given the information that we need to make 3 clusters. It means we are given $K=3$

K-means Clustering Numerical Example

Point	Coordinates
A1	(2,10)
A2	(2,6)
A3	(11,11)
A4	(6,9)
A5	(6,4)
A6	(1,2)
A7	(5,10)
A8	(4,9)
A9	(10,12)
A10	(7,5)
A11	(9,11)
A12	(4,6)
A13	(3,10)
A14	(3,8)
A15	(6,11)

- Randomly choose 3 centroids from the given data.
- Let us consider A2 (2,6), A7 (5,10), and A15 (6,11) as the centroids of the initial clusters

Centroid 1=(2,6) is associated with cluster 1.

Centroid 2=(5,10) is associated with cluster 2.

Centroid 3=(6,11) is associated with cluster 3.

Point	Distance from Centroid 1 (2,6)	Distance from Centroid 2 (5,10)	Distance from Centroid 3 (6,11)	Assigned Cluster
A1 (2,10)	4	3	4.123106	Cluster 2
A2 (2,6)	0	5	6.403124	Cluster 1
A3 (11,11)	10.29563	6.082763	5	Cluster 3
A4 (6,9)	5	1.414214	2	Cluster 2
A5 (6,4)	4.472136	6.082763	7	Cluster 1
A6 (1,2)	4.123106	8.944272	10.29563	Cluster 1
A7 (5,10)	5	0	1.414214	Cluster 2
A8 (4,9)	3.605551	1.414214	2.828427	Cluster 2
A9 (10,12)	10	5.385165	4.123106	Cluster 3
A10 (7,5)	5.09902	5.385165	6.082763	Cluster 1
A11 (9,11)	8.602325	4.123106	3	Cluster 3
A12 (4,6)	2	4.123106	5.385165	Cluster 1
A13 (3,10)	4.123106	2	3.162278	Cluster 2
A14 (3,8)	2.236068	2.828427	4.242641	Cluster 1
A15 (6,11)	6.403124	1.414214	0	Cluster 3

Iteration 1

- Find the Euclidean distance between each point and the centroids.

m

- Based on the minimum distance of each point from the centroids, we will assign the points to a cluster.

$$d = \text{SQRT}(x_2 - x_1)^2 + (y_2 - y_1)^2$$

- In cluster 1, we have 6 points i.e. A2 (2,6), A5 (6,4), A6 (1,2), A10 (7,5), A12 (4,6), A14 (3,8). To calculate the new centroid for cluster 1, we will find the mean of the x and y coordinates of each point in the cluster. Hence, **the new centroid for cluster 1 is (3.833, 5.167).**
- In cluster 2, we have 5 points i.e. A1 (2,10), A4 (6,9), A7 (5,10), A8 (4,9), and A13 (3,10). Hence, **the new centroid for cluster 2 is (4, 9.6)**
- In cluster 3, we have 4 points i.e. A3 (11,11), A9 (10,12), A11 (9,11), and A15 (6,11). Hence, **the new centroid for cluster 3 is (9, 11.25).**

Calculate the distance of each data point from the new centroids. Then, we will assign the points to clusters based on their distance from the centroids.

Point	Distance from Centroid 1 (3.833, 5.167)	Distance from centroid 2 (4, 9.6)	Distance from centroid 3 (9, 11.25)	Assigned Cluster
A1 (2,10)	5.169	2.040	7.111	Cluster 2
A2 (2,6)	2.013	4.118	8.750	Cluster 1
A3 (11,11)	9.241	7.139	2.016	Cluster 3
A4 (6,9)	4.403	2.088	3.750	Cluster 2
A5 (6,4)	2.461	5.946	7.846	Cluster 1
A6 (1,2)	4.249	8.171	12.230	Cluster 1
A7 (5,10)	4.972	1.077	4.191	Cluster 2
A8 (4,9)	3.837	0.600	5.483	Cluster 2
A9 (10,12)	9.204	6.462	1.250	Cluster 3
A10 (7,5)	3.171	5.492	6.562	Cluster 1
A11 (9,11)	7.792	5.192	0.250	Cluster 3
A12 (4,6)	0.850	3.600	7.250	Cluster 1
A13 (3,10)	4.904	1.077	6.129	Cluster 2
A14 (3,8)	2.953	1.887	6.824	Cluster 2
A15 (6,11)	6.223	2.441	3.010	Cluster 2

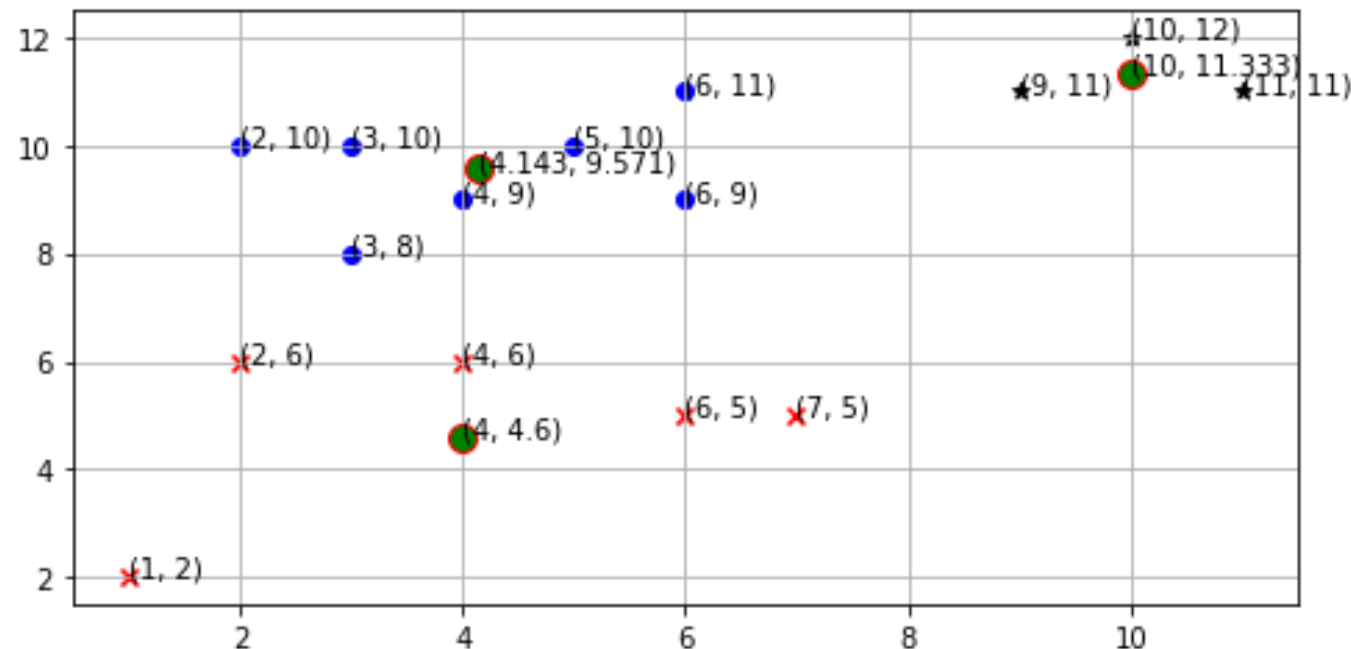
Iteration 2

- In cluster 1, we have 5 points i.e. A2 (2,6), A5 (6,4), A6 (1,2), A10 (7,5), and A12 (4,6). To calculate the new centroid for cluster 1, we will find the mean of the x and y coordinates of each point in the cluster. Hence, **the new centroid for cluster 1 is (4, 4.6).**
- In cluster 2, we have 7 points i.e. A1 (2,10), A4 (6,9), A7 (5,10), A8 (4,9), A13 (3,10), A14 (3,8), and A15 (6,11). Hence, **the new centroid for cluster 2 is (4.143, 9.571)**
- In cluster 3, we have 3 points i.e. A3 (11,11), A9 (10,12), and A11 (9,11). Hence, **the new centroid for cluster 3 is (10, 11.333).**

Point	Distance from Centroid 1 (4, 4.6)	Distance from centroid 2 (4.143, 9.571)	Distance from centroid 3 (10, 11.333)	Assigned Cluster
A1 (2,10)	5.758	2.186	8.110	Cluster 2
A2 (2,6)	2.441	4.165	9.615	Cluster 1
A3 (11,11)	9.485	7.004	1.054	Cluster 3
A4 (6,9)	4.833	1.943	4.631	Cluster 2
A5 (6,4)	2.088	5.872	8.353	Cluster 1
A6 (1,2)	3.970	8.197	12.966	Cluster 1
A7 (5,10)	5.492	0.958	5.175	Cluster 2
A8 (4,9)	4.400	0.589	6.438	Cluster 2
A9 (10,12)	9.527	6.341	0.667	Cluster 3
A10 (7,5)	3.027	5.390	7.008	Cluster 1
A11 (9,11)	8.122	5.063	1.054	Cluster 3
A12 (4,6)	1.400	3.574	8.028	Cluster 1
A13 (3,10)	5.492	1.221	7.126	Cluster 2
A14 (3,8)	3.544	1.943	7.753	Cluster 2
A15 (6,11)	6.705	2.343	4.014	Cluster 2

Iteration 3

- Now, we will calculate the new centroid for each cluster for the third iteration.
- In cluster 1, we have 5 points i.e. A2 (2,6), A5 (6,4), A6 (1,2), A10 (7,5), and A12 (4,6). Hence, the new centroid for cluster 1 is (4, 4.6).
- In cluster 2, we have 7 points i.e. A1 (2,10), A4 (6,9), A7 (5,10), A8 (4,9), A13 (3,10), A14 (3,8), and A15 (6,11). Hence, the new centroid for cluster 2 is (4.143, 9.571)
- In cluster 3, we have 3 points i.e. A3 (11,11), A9 (10,12), and A11 (9,11). Hence, the new centroid for cluster 3 is (10, 11.333).
- Here, you can observe that no point has changed its cluster compared to the previous iteration. Due to this, the centroid also remains constant. Therefore, we will say that the clusters have been stabilized. Hence, the clusters obtained after the third iteration are the final clusters made from the given dataset. If we plot the clusters on a graph, the graph looks like as follows.



Given Dataset

Point	x	y
P1	1	1
P2	2	1
P3	4	3
P4	5	4

Number of clusters: **K = 2**

Initial centroids:

- $C_1 = (1, 1)$
- $C_2 = (5, 4)$

Final Result After One Iteration

Cluster 1

P1 (1,1)

P2 (2,1)

New centroid $\rightarrow$ **(1.5 , 1)**

Cluster 2

P3 (4,3)

P4 (5,4)

New centroid $\rightarrow$ **(4.5 , 3.5)**

- <https://mitu.co.in/wp-content/uploads/2022/04/Ensemble-Learning.pdf>